

UNIVERSITÉ DE SHERBROOKE · ÉCOLE DE GESTION

GIS805

Session 07 / 12

Soirée d'intégration : multi-star, drill-across et réel-vs-cible

Structuration et analyse multidimensionnelle

C

PROFESSEUR

**Prof. Carlos Denner
dos Santos**

Département des systèmes
d'information et méthodes
quantit...

DATE

4 juin 2026

LIEU

Longueuil

HORAIRE

19 h 00 – 22 h 00

Votre séance, en bref

Voici comment se déroule votre soirée — 3 temps forts.

20 min

Multi-star theory + bus matrix

45 min

Sprint 1 : integration night

40 min

Sprint 2 : actual-vs-budget + board report



*Le board peut-il voir ventes, retours,
inventaire et budget dans une seule
vue sans mentir ?*

— Le CEO de NexaMart Group, séance 6

Ce soir : vos étoiles deviennent un entrepôt

Soiree d'integration NexaMart

- Jusqu'ici, vous avez construit chaque étoile independamment
- **Mais le CEO veut une vue consolidee** : ventes vs retours vs budget
- Cela nécessite des dimensions conformes et un drill-across correct
- **Erreur fatale** : joindre deux tables de faits directement

01 La bus matrix

L'outil d'integration le plus important de l'entrepôt

Qu'est-ce que la bus matrix ?

Le plan d'architecture de votre entrepôt

- Colonnes = tables de faits (fact_sales, fact_returns, fact_inventory, fact_budget)
- Lignes = dimensions (dim_date, dim_product, dim_store, dim_customer)
- Case cochée = la dimension est conforme et partagée entre les deux tables
- Case vide = décision à documenter (manquante ou intentionnellement exclue)
- La bus matrix est le document le plus important pour l'intégration

Exemple : la bus matrix NexaMart remplie

Voici a quoi elle ressemble quand elle est complétée

- | fact_sales | fact_returns | fact_inventory | fact_budget
- dim_date | X | X | X | X
- dim_product | X | X | X | X
- dim_store | X | X | X |
- dim_customer | X | X | |
- dim_promo | X | | |
- X = dimension conforme partagee. Vide = pas applicable a ce processus.

❗ Reproduisez ce format dans docs/bus-matrix.md avec VOS tables et VOS dimensions.

Exemple : la bus matrix NexaMart remplie (suite)

- **Remarquez** : dim_date et dim_product sont conformes PARTOUT -- ce sont vos piliers.

Le danger du produit cartésien

Pourquoi on ne joint JAMAIS deux tables de faits

- fact_sales a 10 000 lignes pour le produit X en janvier
- fact_returns a 500 lignes pour le même produit en janvier
- **JOIN direct** : $10\,000 \times 500 = 5\,000\,000$ lignes -- produit cartésien
- Le revenue est multiplié par 500, les retours par 10 000
- **Solution** : le drill-across via les dimensions conformes

FAUX vs CORRECT : le JOIN mortel et le drill-across

Voyez la difference avant d'ecrire une seule ligne

- [FAUX] JOIN direct entre deux tables de faits
- SELECT p.category, SUM(s.line_total) AS rev, SUM(r.refund_amount) AS ref
- FROM fact_sales s
 - JOIN fact_returns r USING (product_key) -- 10 000 × 500 = 5 millions de lignes
 - JOIN dim_product p USING (product_key)
- **GROUP BY 1; -- FAUX** : rev gonfle x500, ref gonfle x10000
- [CORRECT] Deux CTEs separees, meme grain, FULL OUTER JOIN sur les resultats
- WITH sales_agg AS (SELECT ... GROUP BY category, mois),
 - returns_agg AS (SELECT ... GROUP BY category, mois)

❗ Si deux tables de faits apparaissent dans le meme FROM sans CTE : c'est un produit cartesien garanti.

FAUX vs CORRECT : le JOIN mortel et le drill-across (suite)

- `SELECT ... FROM sales_agg s`
 - `FULL OUTER JOIN returns_agg r ON s.category=r.category AND s.mois=r.mois;`

02 Le drill-across

Comparer des métriques de tables différentes sans duplication

Comment fonctionne le drill-across

Deux requêtes, même dimension, même grain

- **Étape 1** : agréger fact_sales par (product, month) → revenue_par_produit
- **Étape 2** : agréger fact_returns par (product, month) → retours_par_produit
- **Étape 3** : joindre les deux resultats sur (product, month)
- **Resultat** : revenue ET retours côte à côte, sans produit cartésien
- **Le grain commun est la clé** : les deux requêtes doivent agréger au même niveau

Drill-across : le squelette CTE complet

Copiez ce patron dans `sql/integration/s07-drill-across.sql`

- WITH
 - sales_agg AS (
 - SELECT p.category,
 - DATE_TRUNC('month', f.order_date) AS mois,
 - SUM(f.line_total) AS revenue,
 - SUM(f.quantity) AS units_sold
 - FROM fact_sales f JOIN dim_product p USING (product_key)
 - GROUP BY 1, 2),
 - returns_agg AS (
 - SELECT p.category,
 - DATE_TRUNC('month', r.return_date) AS mois,
 - SUM(r.refund_amount) AS total_refunds,
 - SUM(r.return_quantity) AS units_returned
 - FROM fact_returns r JOIN dim_product p USING (product_key)
 - GROUP BY 1, 2)
 - SELECT COALESCE(s.category, r.category) AS category,

Réel vs budget

Le drill-across le plus demande par les CFO

- fact_budget est mensuel par catégorie x magasin
- fact_sales est transactionnel (grain plus fin)
- **Pour comparer** : agréger fact_sales au grain de fact_budget
- Joindre sur (catégorie, magasin, mois) -- le grain commun
- $\text{Ecart} = \text{revenue_reel} - \text{budget} \rightarrow$ le CFO veut cette ligne

Grain alignment : le defi de fact_budget

fact_budget.category est un VARCHAR — pas une FK vers dim_product

- **fact_sales** : grain = transaction. Les mesures passent par product_key → dim_product → category
- **fact_budget** : grain = category x magasin x mois (category = VARCHAR brut, pas de FK)
- **Pour joindre** : il faut d'abord agreger fact_sales via dim_product pour obtenir p.category
- WITH actual AS (
 - SELECT p.category, f.store_key, DATE_TRUNC('month', f.order_date) AS mois,
 - SUM(f.line_total) AS actual_revenue
 - FROM fact_sales f JOIN dim_product p USING (product_key) GROUP BY 1, 2, 3)
- SELECT ... FROM actual a FULL OUTER JOIN fact_budget b
 - ON a.category = b.category
 - AND a.store_key = b.store_key
 - AND a.mois = b.budget_month;
- variance = actual_revenue - target_revenue

❗ Erreur frequente : joindre fact_sales a fact_budget sans dim_product → category NULL partout.

Grain alignment : le defi de fact_budget (suite)

- $\text{pct_variance} = \text{ROUND}(100.0 * \text{variance} / \text{NULLIF}(\text{target_revenue}, 0), 1)$

Votre copilote cette semaine

Comment travailler avec votre assistant IA pour le drill-across

✓ PROMPTS QUI MARCHENT

- Pourquoi ne peut-on JAMAIS joindre deux tables de faits directement ?
- Ecris un drill-across : agrege ventes et retours au même grain, puis joins.
- Genere une vue actual-vs-budget avec variance.

✗ PROMPTS À REJETER

- l'assistant joint fact_sales a fact_returns directement (produit cartésien)
- les totaux drill-across ne reconcilient pas avec les totaux individuels

03 Sprint 1 : integration night

Charger 4 tables de faits et vérifier la conformité

Votre mission

45 minutes pour intégrer vos étoiles

- 1. Chargez les 4 tables de faits dans DuckDB
- 2. Vérifiez que dim_date et dim_product sont conformes entre toutes les tables
- 3. Ecrivez un drill-across entre fact_sales et fact_returns
- 4. Documentez votre bus matrix dans docs/bus-matrix.md

❗ Erreur fréquente : JOIN fact_sales TO fact_returns directement. Arrêtez-vous si vous voyez ça.

Sprint 1 : etapes SQL concretes

Fichier cible : `sql/integration/s07-drill-across.sql`

- 0. make generate && make load
 - `SELECT COUNT(*) FROM fact_sales; -- doit etre > 0`
 - `SELECT COUNT(*) FROM fact_returns; -- doit etre > 0`
- **1. Verifiez la conformite** : le meme product_key doit exister dans dim_product
 - `SELECT COUNT(DISTINCT product_key) FROM fact_sales;`
 - `SELECT COUNT(DISTINCT product_key) FROM fact_returns;`
- 2. Copiez le squelette CTE du slide precedent dans `s07-drill-across.sql`
- 3. Verification obligatoire (ajoutez en commentaire avec le resultat observe) :
 - `SELECT SUM(revenue) FROM sales_agg; doit = SELECT SUM(line_total) FROM fact_sales`
- 4. `git add -A && git commit -m 'S07 sprint1 drill-across fonctionnel'`
- 5. Documentez votre bus matrix dans `docs/bus-matrix.md`

❗ Si les totaux ne reconcilient pas → GROUP BY incomplet dans la CTE. Ajoutez mois au GROUP BY.

04 Sprint 2 : réel vs budget

Le rapport que le CFO attend

Sprint 2 : etapes SQL concretes

Fichier cible : sql/integration/s07-actual-vs-budget.sql

- 1. Verifiez les categories (case-sensitive !) :
 - `SELECT DISTINCT category FROM fact_budget ORDER BY 1;`
 - `SELECT DISTINCT category FROM dim_product ORDER BY 1;`
 - Les deux listes doivent correspondre exactement
- 2. **CTE actual** : agreger fact_sales via dim_product (category + store_key + mois)
 - `SELECT p.category, f.store_key, DATE_TRUNC('month', f.order_date) AS mois,`
 - `SUM(f.line_total) AS actual_revenue`
 - `FROM fact_sales f JOIN dim_product p USING (product_key) GROUP BY 1, 2, 3`
- 3. **FULL OUTER JOIN fact_budget ON (category, store_key, budget_month)**
- 4. Ajoutez variance et pct_variance :
 - `variance = actual_revenue - target_revenue`
 - `pct_variance = ROUND(100.0 * variance / NULLIF(target_revenue, 0), 1)`
- 5. **Verif** : `SELECT SUM(actual_revenue) FROM actual; doit = SUM(line_total) FROM fact_sales`
- 6. `git add -A && git commit -m 'S07 sprint2 actual-vs-budget'`

Livrable final

Vue consolidee + board brief

- **Vue SQL** : actual_revenue vs budget par catégorie x magasin x mois
- **Reconciliation** : prouver que les totaux n'incluent pas de duplication
- **Board brief** : répondre a la question du CEO avec evidence multi-table

Soumission et evaluation

Ce qui est attendu avant S08

- **Fichiers** : answers/S07_executive_brief.md + docs/bus-matrix.md
- **SQL** : sql/integration/s07-drill-across.sql + sql/integration/s07-actual-vs-budget.sql
- **Evaluation** : modèle (40%), validation (25%), justification exécutive (20%)
- Trace de processus (10%), reproductibilite (5%)
- **Commitez** : git add -A && git commit -m "S07 integration night" && git push
- Consultez docs/verify-before-pushing.md avant de pousser

Où en sommes-nous ?

Votre parcours en 14 séances

- S01-S04 : Étoile, grain, SCD, dims spéciales [FAIT]
- S05 : Révision + prérequis [FAIT]
- S06 : Examen intra 1 [FAIT]
- >> **S07** : Integration multi-star, bus matrix, drill-across [AUJOURD'HUI]
 - S08 : Dates role-playing, hiérarchies, NULLs
 - S09 : 4 types de tables de faits
 - S10 : Examen intra 2 (S06-S09)
 - S11-S14 : Documentation, défense, au-dela, final

A retenir de ce soir

- 1 JAMAIS joindre deux tables de faits directement -- produit cartésien garanti.
- 2 Drill-across = deux requêtes au même grain, jointes sur les dimensions conformes.
- 3 La bus matrix est le document d'architecture central de l'entrepôt.
- 4 Le grain commun est la clé du réel-vs-budget.
- 5 La semaine prochaine : dates role-playing, hiérarchies et NULLs (S08).